

デジタルツイン技術スタック ケーススタディ: 確立分野の調査

自動運転・自動配膳など、デジタルツインが実用化されている分野の技術スタックを整理する。

1. 自動運転分野

1.1 技術スタック全体像

自動運転におけるデジタルツインは、シミュレーション環境での検証→実世界への展開（Sim2Real）のパイプラインが中心となる。

レイヤー	主要技術
シミュレータ	CARLA (Unreal Engine), NVIDIA DRIVE Sim (Omniverse), LGSVL, SUMO (交通流)
ミドルウェア	ROS / ROS 2
知覚 (Perception)	YOLOv5/v8/v11, PointPillars, CenterPoint, LiDAR-Camera Fusion
地図・標準	ASAM OpenDRIVE, OpenSCENARIO, HD Map
ADSフレームワーク	Autoware, Baidu Apollo
AI/MLフレームワーク	PyTorch, TensorFlow, CUDA, TensorRT
クラウド/エッジ	NVIDIA DGX, AWS, Azure, Baidu Cloud
ハードウェア	NVIDIA Orin / Drive AGX, RTX GPU群, LiDAR, カメラ, レーダー, IMU

1.2 CARLA シミュレータ

概要: オープンソースの自動運転シミュレータ。Unreal Engineベースで物理的に正確な環境を構築可能。

技術スタック:

- ・レンダリングエンジン: Unreal Engine 4/5
- ・センサーシミュレーション: LiDAR, RGB/深度カメラ, GNSS, IMU, レーダー
- ・地図規格: ASAM OpenDRIVE (MathWorks RoadRunnerで作成可能)
- ・ROS連携: CARLA ROS-bridgeによりROS/ROS 2と直接接続
- ・ADSフレームワーク統合: AutowareエージェントおよびConditional Imitation Learningエージェントを同梱
- ・交通流シミュレーション: SUMOとの共同シミュレーション (Co-simulation)
- ・言語: Python 3.8+, C++
- ・推論基盤: CUDA 11.3+, PyTorch 1.11+
- ・ハードウェア要件: RTX 3090 (24GB) 以上のGPU, 12コア以上のCPU

デジタルツインとしての活用事例:

- ・ CARLA-Twin:

2台のCARLAインスタンスを用い、1台を物理世界、もう1台をツイン世界として双方向通信で接続。衝突率95%削減、平均車速2.8倍の改善を実証。

- ・ DTF-Sim: 実交通データからデジタルツイン交通流を再現し、CARLAと共同シミュレーション。

・ Sim2Real DVP: CARLA→並列実行→実世界の3段階で現実ギャップ (Reality Gap) と性能ギャップ (Performance Gap) を段階的に解消するMCRPG手法。

参考リンク:

- ・ [CARLA公式サイト](#)
- ・ [CARLA GitHub](#)
- ・ [CARLA-Twin論文 \(IEEE INFOCOM 2025\)](#)
- ・ [Sim2Real DVPプラットフォーム \(Sensors, 2026\)](#)

1.3 NVIDIA DRIVE Sim / Omniverse

概要:

NVIDIAが構築する、ゲームエンジンではなくシミュレーションエンジンとして設計された自動運転専用プラットフォーム。

技術スタック:

- ・ 基盤: NVIDIA Omniverse (USD: Universal Scene Descriptionベース)
- ・ レンダリング: RTXレイトレーシングによる物理的に正確なセンサーシミュレーション
- ・ センサー: カメラ, LiDAR, レーダーの同時リアルタイムシミュレーション (L2~L5対応)
- ・ 地図: NVIDIA DRIVE Map (道路ネットワークのデジタルツイン)
- ・ データ生成: NVIDIA Cosmos (世界モデルによるセンサーデータ多様化)
- ・ スケーラビリティ: マルチGPU分散コンピューティング対応
- ・ 検証方式: SIL (Software-in-the-Loop) / HIL (Hardware-in-the-Loop)
- ・ シナリオ再構成: AI活用で実走行データからシナリオを自動再構築・編集

デジタルツインとしての特徴:

- ・ 実環境を5cm精度でスキャンし、シミュレーション内に再現 (例: NVIDIA本社周辺17マイルルート)
- ・ 時間帯、天候、交通量をパラメトリックに変更可能
- ・ Mercedes-Benz EQSなど実車両のデジタルツインを使った検証

参考リンク:

- ・ [NVIDIA DRIVE Sim AI Tools](#)
- ・ [NVIDIA Omniverse](#)
- ・ [AV Sensor Simulation](#)

1.4 Baidu Apollo

概要: Baiduが開発するオープンソース自動運転プラットフォーム。L2+~L4の自律走行技術を提供。

技術スタック:

- ・ OS: カスタムLinuxカーネル 4.4.32-apollo (PREEMPT-RTパッチ適用でリアルタイム性確保)
- ・ コアモジュール: ローカライゼーション, パーセプション, プランニング, コントロール
- ・ HD Map: ローカライゼーション用地図 + 車線・信号・横断歩道メタデータ付き計画用地図
- ・ 知覚システム: ビッグデータ+深層学習、OTAアップデートによるモデル継続改善
- ・ データパイプライン: Apollo Loop (オンボード小型AIモデル + クラウド大規模AIモデルによる閉ループデータシステム)
- ・ 開発ツール: Dreamview Plus (可視化・パネルカスタマイズ・リソースセンター)
- ・ 計算基盤: NVIDIA Orin X SoC × 2 (508 TOPS), Qualcomm 8295, CUDA 11.8
- ・ センサー: 高光束LiDAR × 2 (検出距離180m), 8MPカメラ × 7, 3MPサラウンドビューカメラ × 4, レーダー × 5, 超音波センサー × 12

参考リンク:

- ・ [Apollo GitHub](#)
- ・ [Apollo Developer Portal](#)
- ・ [Apollo vs Autoware比較論文](#)

1.5 Autoware

概要: 世界初のオールインワン・オープンソース自動運転ソフトウェアスタック。名古屋大学とTier IVが開発開始、現在はAutoware Foundationが管理。

技術スタック:

- ・ ミドルウェア: ROS 2 (リアルタイム性、QoS、分散アーキテクチャ対応)
- ・ コア機能: ローカライゼーション (LiDAR, GNSS, IMU), 知覚 (物体検出・追跡: LiDAR, カメラ, レーダー), 経路計画, 車両制御
- ・ 地図: HD Map, 車両-インフラ間通信 (V2I) 対応
- ・ リポジトリ構成:
 - Autoware Core: 安定版ROSパッケージ
 - Autoware Universe: 実験的・最先端ROSパッケージ
- ・ ML基盤: AWML (Autoware ML-based Perception Framework)
- ・ ハードウェア要件: 最小8コアCPU, 16GB RAM (スケーラブル設計)
- ・ センサー非依存: LiDAR + カメラ融合が標準だが、センサー構成は自由に変更可能
- ・ ライセンス: Apache 2.0 (商用利用可)

参考リンク:

- ・ [Autoware公式サイト](#)
- ・ [Autoware GitHub](#)
- ・ [AWML論文](#)

2. 自動配膳ロボット分野

2.1 技術スタック全体像

配膳ロボットは、屋内環境での自律移動を前提とし、SLAM + LiDAR + 深度カメラ + マルチロボット協調が技術の柱となる。

レイヤー	主要技術
ナビゲーション	SLAM（レーザーSLAM, ビジュアルSLAM）, タグナビゲーション
SLAMアルゴリズム	GMapping, Hector SLAM, Cartographer, RTAB-MAP, slam_toolbox
経路計画	A*（グローバル）, DWA（ローカル障害物回避）
センサー	2D/3D LiDAR, RGB-Dカメラ, IMU, エンコーダ, 超音波, 重量センサー
知覚AI	YOLOv7-Tiny（軽量モデル）, 3D物体検出
ミドルウェア	ROS / ROS 2（研究用）, 独自ミドルウェア（商用製品）
マルチロボット制御	分散型アドホックネットワーク通信,
クラウド基盤	各社独自クラウド管理プラットフォーム
エッジコンピューティング	Raspberry Pi 4, ESP32 S3, ARM系SoC

2.2 Pudu Robotics BellaBot

概要：世界で最も普及しているネコ型配膳ロボット。700以上の特許を保有。

コア技術4本柱：

1. SLAM：レーザーSLAM + ビジュアルSLAMの両方をサポート
2. 無指向性3D障害物回避：前方検出角度192.64°、RGB-Dカメラ × 3台、0.5秒で停止応答
3. PUDU SCHEDULER:
分散型アドホックネットワーク通信によるマルチロボット協調制御。各ロボットが同一ネットワーク内の任意のロボットと直接通信
4. PUDU CLOUD:
ビジネス管理、自動運用・保守、クラウドインテリジェントサービスの統合プラットフォーム

ハードウェア仕様：

- ・4段トレイ、最大40kg積載
- ・1充電で12～24時間稼働
- ・独立懸架型サスペンション（段差・凹凸対応）
- ・速度：0.3～1.2 m/s

最新動向（2025年～）：

- ・PUDU D5：イヌ型4足歩行ロボット。120° 魚眼カメラ × 4（360° 全方位認識）+ LiDAR × 2（前後）
- ・BellaBot 工業用バージョン：工場・倉庫向けにカスタマイズ

参考リンク：

- ・ [BellaBot公式（HCI）](#)
- ・ [Pudu Robotics](#)
- ・ [BellaBot技術解説（SEIKO）](#)

2.3 Bear Robotics Servi

概要：元GoogleエンジニアがシリコンバレーでCreate。SoftBank出資。日米韓で展開。

技術スタック：

- ・ センサー：3Dカメラ + LiDAR（前方死角なし、最短60cm幅通過）
- ・ ナビゲーション：SLAM技術（天井タグ不要の自律走行）
- ・ 障害物回避：マルチカメラ + アルゴリズムによる死角ゼロ回避
- ・ 自動復帰：重量センサーで料理取り出しを検知し自動帰還
- ・ 運用監視：24時間365日クラウドモニタリング&メンテナンス

Servi Plus追加機能：

- ・ 3面LEDマトリックスディスプレイ
- ・ 6カ国語対応サウンドシステム
- ・ 状況に応じた代替ルート・停止場所の自動決定

参考リンク：

- ・ [SoftBank Robotics Servi](#)
- ・ [Bear Roboticsについて（自動運転ラボ）](#)

2.4 配膳ロボットにおけるROS活用（研究事例）

商用製品は独自ミドルウェアを使用するケースが多いが、研究・プロトタイピングではROSベースが主流。

典型的なROSベース配膳ロボットの構成：

アプリケーション層	
注文管理 / テーブル割当 / UI	
ROS 2 ミドルウェア	
Navigation2 / SLAM / TF2 / DDS	
知覚	経路計画
LiDAR Scan	A*（グローバル）
RGB-D	DWA（ローカル）
AMCL/EKF	Costmap
ハードウェア抽象化層	
Motor Driver / Encoder / IMU	
エッジコンピュータ	
Raspberry Pi 4 / Jetson Nano / ESP32	

参考リンク：

- ・ [ROS-Based Delivery Robot for Restaurant \(2025\)](#)
- ・ [ROS-Based Navigation Architectures \(PMC\)](#)
- ・ [配膳ロボットの仕組み解説（双日ロボティクス）](#)

3. 両分野の技術スタック比較

観点	自動運転	自動配膳
環境	屋外（道路・高速道路）	屋内（レストラン・倉庫）
速度域	0～130 km/h	0.3～1.2 m/s
センサー構成	LiDAR + カメラ × 多数 + レーダー + 超音波 + GNSS + IMU	LiDAR + RGB-Dカメラ × 数台 + エンコーダ
計算基盤	NVIDIA Orin (508 TOPS), GPU	Raspberry Pi, ARM SoC, ESP32
SLAM	Huber Map + ローカライゼーション（事前地図前提）	リアルタイムSLAM（動的環境対応）
シミュレータ	CARLA, DRIVE Sim, LGSVL	Gazebo（研究用）, 商用は実機テスト中心
ミドルウェア	ROS 2 / 独自	ROS 2（研究） / 独自（商用）
マルチエージェント	V2V/V2I通信、交通流連携	アドホックネットワーク、クラウドスケジューラ
安全基準	ISO 26262, SOTIF, UL 4600	独自安全基準、CE/FCC認証
デジタルツイン活用度	高（Sim2Real、シナリオ再構成、大規模検証）	中（店舗マッピング、フリート管理）
市場成熟度	L2+商用化済み、L4実証段階	商用化済み（世界数万台稼働）

4. デジタルツイン技術スタック設計への示唆

両分野から得られる、デジタルツイン技術スタック設計の共通パターン：

4.1 共通アーキテクチャパターン

1. センサーフュージョン層：複数種類のセンサーデータを統合し、環境の高精度な3D表現を構築
2. SLAM/マッピング層：物理環境のデジタルツインを構築・更新する基盤技術
3. ミドルウェア層：ROS/ROS 2が事実上の標準。モジュール間通信、データ配信を担当
4. シミュレーション層：物理世界のデジタルツインを用いたテスト・検証環境
5. クラウド/エッジ連携層：リアルタイムデータ収集、モデル更新、フリート管理

4.2 技術選定のポイント

- ・精度 vs コスト：自動運転は高精度・高コスト（LiDAR数百万円級）、配膳は低コスト・実用十分な精度
- ・リアルタイム性：自動運転はミリ秒単位の応答が必要（PREEMPT-RTカーネル）、配膳は秒単位で十分
- ・Sim2Real：自動運転では不可欠（Reality Gap/Performance Gapの段階的解消）、配膳では実機テスト中心
- ・スケーラビリティ：マルチGPU（自動運転）vs エッジコンピューティング（配膳）